Concurrency Control in Transactional Databases

A study guide for database systems courses

1. Why concurrency control exists

A transaction is a sequence of reads and writes that the database treats as a single logical unit. The ACID properties promise that a transaction is atomic, leaves the database consistent, is isolated from other transactions, and is durable once committed. This guide is about isolation: what happens when many transactions run at the same time.

Running transactions one after another would be simple and correct, but it wastes hardware. Real systems interleave operations from many transactions. The job of concurrency control is to allow as much interleaving as possible while producing results that are equivalent to some serial order of the same transactions. A schedule with that property is called serializable.

Without concurrency control, several well-known anomalies appear:

- A dirty read happens when a transaction reads data written by another transaction that has not yet committed.
- A lost update happens when two transactions read the same value, both modify it, and the second write silently overwrites the first.
- A non-repeatable read happens when a transaction reads the same row twice and sees different values because another transaction committed in between.
- A phantom happens when a query returns a different set of rows the second time it runs because another transaction inserted or deleted matching rows.
- Write skew happens when two transactions read overlapping data, make decisions based on what they read, and then update different rows, producing a state that neither transaction would have allowed on its own.

Concurrency control mechanisms fall into two broad families. Pessimistic concurrency control assumes that conflicts are likely and prevents them before they happen, usually with locks. Optimistic concurrency control assumes that conflicts are rare, lets transactions proceed without blocking, and checks for conflicts before a transaction commits. Multiversion concurrency control, discussed in section 4, is often combined with either family.

2. Pessimistic concurrency control with locks

Pessimistic schemes protect data by requiring a transaction to acquire a lock before it accesses an item. A shared lock allows reading and can be held by many transactions at once. An exclusive lock allows writing and is incompatible with every other lock on the same item. A transaction that requests an incompatible lock must wait until the current holder releases it.

Locking alone does not guarantee serializability. The classic protocol that does is two-phase locking (2PL). Under 2PL every transaction has a growing phase, in which it may acquire locks but not release any, followed by a shrinking phase, in which it may release locks but not acquire new ones. Any schedule produced by 2PL is conflict serializable.

Basic 2PL still allows cascading aborts: if a transaction releases an exclusive lock before it commits, another transaction may read the uncommitted value and must be aborted if the first transaction later aborts. Strict two-phase locking avoids this problem by holding all exclusive locks until the transaction commits or aborts. Most lock-based database systems implement strict 2PL or a close variant.

Deadlocks

Because transactions wait for each other, locking can produce deadlocks, in which two or more transactions each wait for a lock held by another. Systems handle deadlocks in two ways.

Deadlock detection builds a wait-for graph, in which an edge from T1 to T2 means that T1 waits for a lock held by T2. A cycle in this graph is a deadlock, and the system breaks it by aborting one of the transactions in the cycle.

Deadlock prevention uses transaction timestamps to decide in advance who may wait. In the wait-die scheme, an older transaction that requests a lock held by a younger one waits, while a younger transaction that requests a lock held by an older one is aborted. In the wound-wait scheme, an older transaction that requests a lock held by a younger one wounds the younger transaction, which is aborted, while a younger transaction that requests a lock held by an older one waits. Both schemes guarantee that no cycle of waiting transactions can form.

Lock granularity

Locks can protect whole tables, pages or individual rows. Fine-grained row locks allow more concurrency but cost more to manage. Intention locks let a transaction announce at the table level that it holds or plans to hold locks on rows inside the table, so that a request for a table-level lock can be checked without inspecting every row lock.

3. Optimistic concurrency control

Optimistic concurrency control was introduced by H. T. Kung and John T. Robinson in their 1981 paper "On Optimistic Methods for Concurrency Control". The central observation is that in many workloads, transactions rarely touch the same data, so the cost of acquiring locks is paid even when no conflict would have occurred.

An optimistic transaction executes in three phases:

- In the read phase, the transaction reads values from the database and writes its changes to a private workspace. Other transactions cannot see these changes.
- In the validation phase, the system checks whether the transaction conflicts with transactions that committed while it was running.
- In the write phase, a transaction that passed validation copies its private changes into the database. A transaction that failed validation is aborted and usually restarted.

A common validation method is backward validation. The validating transaction compares its read set with the write sets of the transactions that committed after it started. If any item it read was written by one of those transactions, the values it based its decisions on may be stale, so it must abort.

In application code the same idea often appears as a version column. A row carries a version number, an update succeeds only if the version is unchanged since the row was read, and the version is incremented on every successful update. If the update affects zero rows, the application knows another writer got there first and retries.

Costs of being optimistic

Optimistic schemes never block and never deadlock, which makes them attractive for read-heavy workloads and for short transactions with little overlap. Their weakness appears under high contention. When many transactions update the same items, a large fraction of them fail validation, and all the work they performed before validation is wasted. Retries then add more load to exactly the data that is already contended, so throughput can collapse instead of degrading gracefully. Long transactions are especially exposed, because the longer a transaction runs, the more concurrent commits it must be validated against.

4. Multiversion concurrency control and snapshot isolation

Multiversion concurrency control (MVCC) keeps several versions of each data item. A write creates a new version instead of overwriting the old one, and each transaction reads the versions that were committed when its snapshot was taken. Because readers see an older version instead of waiting, readers do not block writers and writers do not block readers. MVCC is used by PostgreSQL, Oracle, MySQL's InnoDB engine and many other systems.

The isolation level most closely associated with MVCC is snapshot isolation. Each transaction reads from a consistent snapshot, and when two concurrent transactions write the same item, only one of them may commit; this rule is often called first-committer-wins. Snapshot isolation prevents dirty reads, non-repeatable reads and lost updates, but it is not serializable.

Write skew under snapshot isolation

The standard example of an anomaly that snapshot isolation allows is write skew. Imagine a hospital rule that at least one doctor must be on call. Two doctors are on call, and both request to go off call at the same moment. Each transaction reads the current on-call list from its snapshot, sees that another doctor is still on call, and removes only its own doctor. The two transactions write different rows, so first-committer-wins does not detect a conflict. Both commit, and no doctor is on call.

Serializable snapshot isolation

Serializable snapshot isolation (SSI), described by Michael Cahill, Uwe Röhm and Alan Fekete in 2008, keeps the non-blocking reads of snapshot isolation but tracks read-write dependencies between concurrent transactions. When it detects a dangerous structure of dependencies that could lead to a non-serializable outcome, it aborts one of the transactions involved. PostgreSQL has used SSI to implement its SERIALIZABLE isolation level since version 9.1. Applications that run at this level must be prepared to retry transactions that fail with a serialization error.

SSI behaves optimistically in one important way: it lets transactions run without blocking and resolves conflicts by aborting rather than waiting. Its abort rate therefore rises with contention, just like classic optimistic concurrency control, although it typically aborts fewer transactions than a validation scheme that compares complete read and write sets.

5. Choosing an approach

Neither approach is universally faster. The right choice depends on how often transactions conflict and how expensive an abort is.

- Pessimistic locking performs well when contention is high, because waiting for a lock is cheaper than repeatedly executing a transaction that will be aborted. Its costs are lock management overhead, blocking, and the need to handle deadlocks.
- Optimistic concurrency control performs well when contention is low and transactions are short, because it avoids lock overhead and never blocks. Its cost is wasted work when validation fails, which grows quickly as contention increases.
- Multiversion schemes are a strong default for mixed workloads with many readers, because long-running reports do not block updates. Applications still need to choose an isolation level and handle serialization failures.

Scalability considerations

On large multi-core machines, a centralised lock manager can become a scalability bottleneck, because every lock request updates shared data structures that many cores contend for. Optimistic and multiversion designs reduce this shared state, which is one reason they are popular in modern in-memory databases. In distributed databases, locks that span machines require extra network round trips, so distributed systems often combine multiversioning with optimistic validation or with timestamp ordering.

Summary

- Concurrency control aims for serializable outcomes while allowing interleaving.
- Two-phase locking guarantees conflict serializability; strict 2PL also prevents cascading aborts.
- Deadlocks are handled by detection with a wait-for graph or prevention with wait-die and wound-wait.
- Optimistic concurrency control runs a read phase, a validation phase and a write phase, and aborts on conflict.
- MVCC lets readers and writers proceed without blocking each other; snapshot isolation is not serializable because it allows write skew.
- Serializable snapshot isolation detects dangerous dependency structures and aborts one transaction to restore serializability.